`std::const_iterator` often produces an unexpected type

Document #:	P2836R0
Date:	2023-03-20
Project:	Programming Language C++
Audience:	Ranges Study Group (SG9) Library Evolution Working Group (LEWG) Library Working Group (LWG)

Reply-to:	Christopher Di Bella <cjdb@google.com>

Contents

- 1 Introduction
- 2 Acknowledgements
- 3 Problem
  - 3.1 Potential backwards compatibility issues
  - 3.2 Template instantiation issue
  - 3.3 Didn't P2278 note some complexities?
- 4 Potential resolutions
  - 4.1 `const_iterator_for`
    - 4.1.1 Alternative names
  - 4.2 Preferred resolution
    - 4.2.1 Impact on ABI
  - 4.3 Alternative resolution
  - 4.4 Changes to `ranges::cbegin`
- 5 Why is this being identified *now*?
- 6 Implementation experience
- 7 References

§ 1 Introduction

`const_iterator<T>` is too aggressive in what it turns into a `basic_const_iterator<T>`, and it creates inconsistencies. This proposal scales it back.

§ 2 Acknowledgements

Thank you to Richard Smith, Janet Cobb, Nicole Mazzuca, and David Blaikie for providing feedback on the text and design of the proposed resolution.

Thank you to Nicole and Stephan T. Lavavej (STL) for reviewing the CL that implements the proposed fix in my Microsoft/STL fork.

§ 3 Problem

[P2278R4] defined `const_iterator<int*>` to be `basic_const_iterator<int*>` instead of `int const*`, because it seeks to simplify the implementation of esoteric ranges, even though this comes at the expense of more conventional use-cases. As a result, determining the corresponding "const iterator" for an iterator is context-dependent and inconsistent, which is surprising to both the author, some experts, and even implementers. Consider the following snippet of code:

```
using V = std::vector<int>;
auto v = V();

using I1 = decltype(ranges::cbegin(v));
using I2 = ranges::const_iterator_t<V>;
static_assert(std::same_as<I1, I2>);
```

That static assert fails because `I1` is the type `V::const_iterator`, while `I2` has the type `basic_const_iterator<V::iterator>`. While the pointer situation is surprising, there is some wiggle room in arguing that C++'s standard library being somewhat quirky. This crosses that line because `ranges::iterator_t<R const>` and `ranges::const_iterator_t<R>` ought to be analogous: it is fairly reasonable for someone to conclude that because `decltype(std::as_const(vec).begin())` and `decltype(vec.cbegin())` are the same type, so too should `decltype(ranges::begin(std::as_const(vec)))` and `decltype(ranges::cbegin(vec))`.

A third example:

```
using S = ranges::subrange<V::iterator>;
auto s = S(v.begin(), v.end());

using I3 = ranges::const_iterator_t<S>;
static_assert(std::same_as<I1, I3>);
```

Again, this fails because `I3` ends up with the same type as `I2`.

I think all of these situations are far more likely to be encountered (and more common) than P2278's `zstring` example. Since `zstring` has an esoteric implementation, it should equally have an esoteric solution.

§ 3.1 Potential backwards compatibility issues

This problem was discovered while writing a test for `ranges::const_iterator_t`.

```
class always_const_range {
public:
  int* begin() const;
  int* end() const;
};
```

It was surprising to me that `ranges::const_iterator_t<always_const_range>` returned anything other than a pointer, which is the case for C++20. The type does change regardless, but changing from a pointer to a class template is a much larger change that can affect specialisations and possibly even some overload resolution.

Another potential problem is this:

```
class a_range {
  my::vector<int>::iterator begin() const;
  my::vector<int>::iterator end() const;
};

void a_function(my::vector<int>::const_iterator);

// ...

// Works in C++20, breaks in C++23.
auto const r = a_range();
a_function(ranges::cbegin(r));
```

In the above code, `my::vector::iterator` can be implicitly converted to `my::vector::const_iterator` as has been a convention since at least C++11. Since `basic_const_iterator` doesn't appear to have a conversion operator to `my::vector::const_iterator`, however, it's not necessarily possible for `a_function` to be called without making an intrusive change to the call site (we can tack on `.base()`) or `my::vector::const_iterator`. In cases where the user isn't also the owner, this mightn't be possible.

§ 3.2 Template instantiation issue

Here's how template instantiation is affected.

```
bool c1(std::vector<int> const& v, int const value)
{
              //      calls      ranges::find<vector<int>::const_iterator,
        vector<int>::const_iterator, int>
  return ranges::find(v, value) != v.end();
```

```
}

bool c2(std::vector<int> const& v, int const value)
{
            //      calls      ranges::find<vector<int>::const_iterator,
        vector<int>::const_iterator, int>
  return ranges::find(v.cbegin(), v.cend(), 1) != v.end();
}

bool c3(std::vector<int> const& v, int const value)
{
     //   calls   ranges::find<basic_const_iterator<vector<int>::iterator>,
        basic_const_iterator<vector<int>::iterator>, int>
     return   ranges::find(ranges::cbegin(v),   ranges::end(v),   value)   !=
        ranges::cend(v);
}
```

Despite `c1` and `c2` both making a call to a standardised entity named `cbegin`, they're getting back different types, and thus instantiating different specialisations of `ranges::find`. Not only is it surprising, but it will also lead to increased program sizes and compilation slowdowns.

§ 3.3 Didn't P2278 note some complexities?

P2278 talks about adding complexity to `zstring` by introducing a `const_zsentinel` that opens itself up to being comparable with arbitrary pointers, and that by going down this route we'd need to introduce a `const_iterator_cast` since `const_cast` only applies to pointers in this context. P2278 doesn't expand on why that's not desirable, but it's probably safe to conclude that it's for the same reasons we consider `const_cast` to be icky. At least, that's why I don't like it: we shouldn't be giving folks more opportunities to cast away constness.

P2278 also talks about putting the burden on `make_const_iterator`, which can intelligently discern between already constant iterators, pointers, and other types. It argues that this design is more complex and implies it's harder to understand. While this does make it easy to understand in a vacuum, the above cases have identified a variety of issues that can crop up in practical code, leading to potential confusion at point-of-use.

§ 4 Potential resolutions

There are two ways in which we can fix this issue. Both of them require introducing an opt-in mechanism for detecting an iterator's corresponding `const_iterator`. This can be detected using an associated type such as `const_iterator_for` below.

§ 4.1 `const_iterator_for`

```
namespace std {
  template<input_iterator>
  struct const_iterator_for {};
```

```
  template<input_iterator I>
  struct const_iterator_for<const I> : const_iterator_for<I> {};

  template<input_iterator I>
  requires is_pointer_v<I>
   struct const_iterator_for<I> { using type = const remove_pointer_t<T>*;
         };

  template<input_iterator I>
  requires requires { typename I::const_iterator_for; }
  struct const_iterator_for<I> {
    using type = typename I::const_iterator_for;
  };

  template<input_iterator I>
  requires requires { typename const_iterator_for<I>::type; }
  using const_iterator_for_t = typename const_iterator_for<I>::type;
}
```

Then, for example, `std::vector<int>::iterator` could identify that `std::vector<int>::const_iterator` is its `const_iterator` counterpart by adding `const_iterator_for` as a member-alias. The formal wording for `const_iterator_for` requires touching all the iterators in the standard library, which is a laborious task. This will be added once there's general LEWG approval for the paper (and will be added no later than the weekend after discussion).

A more conservative option would be for `const_iterator_for` to be an exposition-only type that works for pointers, but not class types. There are two reasons to consider this: it's more clearly within the scope of a DR, and it doesn't require introducing a new customisation mechanism that impacts the standard library and user code. This is a cost/benefit trade-off: a large portion of the benefits are concerned with handling pointers better, while the cost comes from handling the other cases. This is a halfway measure that is worth considering if folks find the full solution too much for a retroactive fix.

§ 4.1.1 Alternative names

The original name for `const_iterator_for` was `const_iterator_counterpart`. Another potential name might be `const_iterator_is`.

§ 4.2 Preferred resolution

The preferred resolution would be to rework `const_iterator` so it only uses `basic_const_iterator` when `const_iterator_for_t<I>` isn't valid. The following change should be made to [const.iterators.alias]:

```
  template<input_iterator I>
    using const_iterator = see below;
```

Result: If `I` models *`constant-iterator`*, `I`. Otherwise, `basic_const_iterator<I>`.

1 *Result*: If:

(1.1) — `I` models *`constant-iterator`*, `I`;

(1.2) — Otherwise, if `const_iterator_for_t<I>` is well-formed, `const_iterator_for_t<I>`;

(1.3) — Otherwise, `basic_const_iterator<I>`.

4.2.1 Impact on ABI

STL from the Microsoft/STL team has noted that changing `std::const_iterator` now will be an ABI break because it tries to revise a conscious design choice, and isn't personally convinced that it's worth doing. Nicole pointed out that it's not a "full" ABI break since Microsoft hasn't yet released an ABI-stable version. My understanding is that this means that we're not yet at the point-of-no-return, although that date is fast approaching.

4.3 Alternative resolution

An alternative resolution is to add `explicit(false) operator const_iterator_for_t<I>() const` to `basic_const_iterator`. This is a more conservative option that I'm less partial to due to it not solving all the issues, but at least it means that `basic_const_iterator` can play with const iterators.

4.4 Changes to `ranges::cbegin`

P2278 raises the concern that `cbegin` shouldn't produce an iterator that is incompatible with the sentinel produced by `cend`. This is something that I absolutely agree with. Any change to the status quo should not result in an incompatibility between the two. Although `ranges::cbegin(r)` can be evaluated even when `ranges::end(r)` can't be, `ranges::cbegin(r)`'s behaviour is tied to whether or not `ranges::end(r)` is allowed, and what it returns. That is, we only map from `R` to `R const` if `R const` is a constant range when `R` is not: being able to detect that `R const` is a constant range depends on `ranges::end(r)` being well-formed. A potential way to skirt the issue at the start of this paragraph is to change `ranges::cbegin` to also do the same thing for `ranges::end`, except when `ranges::end(r)` produces a sentinel already compatible with `const_iterator<I>`.

Changes to [range.access.cbegin]:

1 The name `ranges::cbegin` denotes a customization point object ([customization.point.object]). Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:

(1.1) — If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is false, `ranges::cbegin(E)` is ill-formed.

(1.2) — Otherwise, let `U` be `ranges::begin(`*`possibly-const-range`*`(t))`. If neither `ranges::end(`*`possibly-const-range`*`(t))` nor `ranges::end(t)` are a valid expressions whose respective types model `sentinel_for<I>`, then `ranges::cbegin(E)` is ill-formed.

(1.3) — Otherwise, `ranges::cbegin(E)` is expression-equivalent to `const_iterator<decltype(U)>(U)`.

§ 5 Why is this being identified *now*?

This originally started out as a defect report to LWG that was issued on 2023-02-09, while I was implementng a C++23-friendly `ranges::cbegin`. I also investigated whether there would be a second round of NB ballot comments, but it seems that the committee is quickly moving toward an FDIS ballot.

This is incredibly late in the game, but the only implementation to have shipped `std::const_iterator` is either Visual Studio 2022 17.5 or 17.6, both of which are 2023 releases (the feature was in review from August through October 2022 according to Microsoft/STL#3043). This means that there was little implementation experience during the C++23 cycle, and next to no deployment experience at scale within that timeframe. Avoiding this in the future, however, should be discussed in a separate paper[1]. We should aim to fix any identified issues as quickly as possible, and also work out a way to prevent this from happening in any future iterations of C++ standardisation.

§ 6 Implementation experience

The above design has been integrated into a Microsoft/STL branch and has been reviewed by two people on the Microsoft/STL team.

§ 7 References

[P2278R4] Barry Revzin. 2022-06-17. cbegin should always return a constant iterator. https://wg21.link/p2278r4

---

1. The irony of this paper's proposed fixes having at most as much implementation and deployment experience as P2278R4 is not lost on me.↩